

Domain Modeling & UML Class Diagram

Student Study Edition · Golden Master design · 14 structured lessons with original course visuals, examples, activities, and study aids.

01 · Domain Modeling & UML Class Diagram

02 · Why Models and Domain Modeling Matter

03 · The Five-Step Domain Modeling Process

04 · OO Foundations: Objects, Classes, Attributes & Operations

05 · UML Class Notation & Types

06 · Discovering Classes: Object or Attribute?

TOPIC 07 · STUDY MAP

Contents

Use this map to move through the topic in the same lesson order as The Educator course pages.

01	Domain Modeling & UML Class Diagram	3
02	Why Models and Domain Modeling Matter	6
03	The Five-Step Domain Modeling Process	11
04	OO Foundations: Objects, Classes, Attributes & Operations	14
05	UML Class Notation & Types	16
06	Discovering Classes: Object or Attribute?	18
07	Inheritance, Generalization & Polymorphism	20
08	Association, Roles & Direction	22
09	Multiplicity & Relationship Constraints	24
10	Aggregation vs Composition	26
11	Choosing the Right Class Relationship	28
12	Worked Example: Online Program Application Domain	31
13	Extra Practice Examples	35
14	Review, Agile Modeling & Exam Checklist	39

TOPIC 07 · LESSON 01 · 1 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 01

Domain Modeling & UML Class Diagram

Before designing software classes, build a shared conceptual picture of the **problem world**: the important concepts, their properties, and the relationships between them.

LEARNING FOCUS

Explain domain modeling as conceptualization and connect domain concepts, properties, and relationships to a UML class diagram.

PRACTICE LAB · TOPIC 07**Mission 07 — Model the Domain Before You Model the Software**

Model EventSphere across 14 stages: discover domain concepts, separate classes from attributes, read UML notation, diagnose inheritance and multiplicity errors, distinguish association, aggregation and composition, and approve the stronger domain model.

VISUAL ANCHOR**Reality → shared concepts → UML****Application domain**

The real-world area the system is about.

**Concepts**

Important kinds of things the team must understand.



Properties

Attributes that describe those concepts.



Relationships

How concepts relate in the domain.

CORE LECTURE

What domain modeling means

- ✓ A model is a conceptual representation that lets a team communicate a shared perception of a piece of reality.
- ✓ Domain modeling is a **conceptualization process** that helps the development team understand the application domain.
- ✓ A domain model defines application-domain concepts using **classes, attributes, and relationships**.
- ✓ It supports communication among team members and, in some cases, customers/users.
- ✓ It provides a basis for design, implementation, and maintenance.
- ✓ For domain modeling, represent the result as a **UML class diagram without operations**.

THE TOPIC JOURNEY

What you will learn

**Why models?**

Shared understanding across different viewpoints.

**Five-step process**

Collect → brainstorm → classify → visualize → review.

**OO concepts**

Object, class, attribute, operation, encapsulation.

**UML notation**

Class compartments and type syntax.

**Class discovery**

Object or attribute?

**Relationships**

Inheritance, association, multiplicity, aggregation, composition.

REMEMBER

DOMAIN MODELING ANALYZES THE PROBLEM WORLD—NOT THE SOFTWARE SOLUTION. The purpose is to make domain knowledge explicit and reviewable before later design decisions.

TOPIC 07 · LESSON 02 · 2 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 02

Why Models and Domain Modeling Matter

Teams need a shared representation because people perceive the same reality differently.

LEARNING FOCUS

Explain why software teams model a domain before design and how a domain model creates shared understanding.

CORE LECTURE

What is a model?

A model is a **conceptual representation** of something: a schematic description that captures known or inferred properties so the subject can be communicated and studied.

- ✓ People perceive the same reality differently because their backgrounds and viewpoints differ.
- ✓ Team members and users therefore need a shared representation for communicating their perceptions and design ideas.
- ✓ Models remain useful during enhancement maintenance because they preserve an inspectable view of the system and its domain.

LECTURE VISUAL · MULTIPLE VIEWPOINTS

One reality can produce different mental models

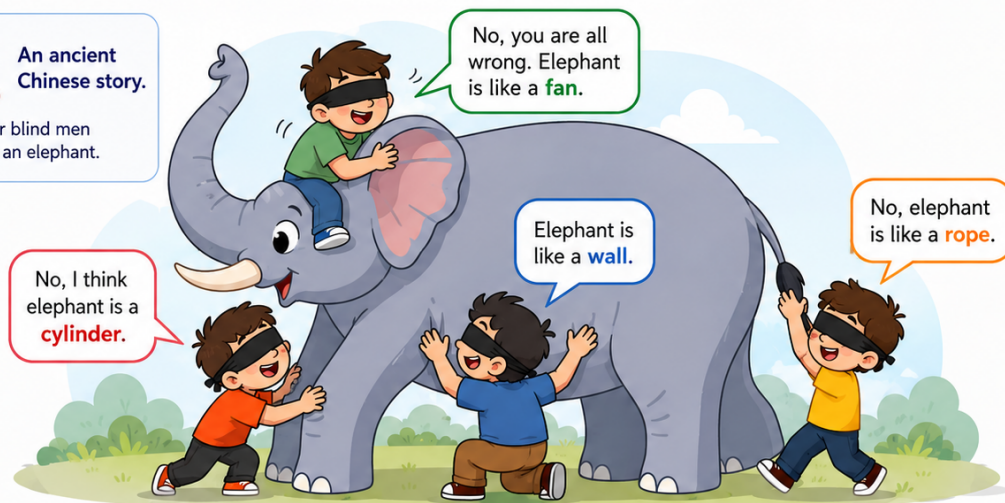


Why Do We Need Model? (1/3)



An ancient Chinese story.

How four blind men perceive an elephant.



We perceive the world **differently** due to differences in **backgrounds** and **viewpoints**. Modeling facilitate **collective** understand of the application.



Shared Understanding
Bridges different perspectives.



Clear Communication
Models simplify complex ideas.



Better Decisions
Enables analysis and evaluation.



Reduced Risk
Helps discover issues early.

Copyright © The McGraw-Hill Companies, Inc.

Different viewpoints

Background and role shape what each person notices.

Why Do We Need Model? (2/3)



Modeling helps everyone share the same understanding!

Benefits:

- ✓ Clear Vision
- ✓ Better Discussion
- ✓ Shared Knowledge
- ✓ Smarter Design



1

Because the team members and the users need to **communicate** their **perception** about a piece of reality.



2

A model facilitates the team members and users to **communicate** their perception and **design ideas**.



Models build a **common understanding** and turn ideas into **clear designs**!

Shared communication

A model gives users and developers something concrete to inspect together.

Why Do We Need Models? (3/3)



Because we need models during the maintenance phase to perform **enhancement maintenance**.



Easier to Understand the System



Supports Maintenance and Evolution



Reduces Cost and Effort



Improves System Reliability

Copyright © The McGraw-Hill Companies, Inc.

9

Long-term value

The model supports understanding beyond initial development.

DOMAIN MODELING · WHAT / WHY / HOW

Domain modeling turns understanding into an explicit artifact

What?

A process that helps the team understand the application and its domain and establish common understanding.

Why?

Software engineers work in different domains and come from different backgrounds, so they need domain knowledge and a shared perception.

How?

Collect domain information, brainstorm and classify concepts, then visualize the knowledge using a UML class diagram.

A domain model defines application-domain concepts in terms of **classes, attributes, and relationships**. It helps analysts understand the domain, improves communication with team members and customers, and provides a basis for design, implementation, and maintenance.

REMEMBER

For domain modeling, the UML class diagram represents domain concepts and relationships *WITHOUT SHOWING OPERATIONS*. The goal is understanding the problem world, not designing software classes prematurely.

TOPIC 07 · LESSON 03 · 3 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 03

The Five-Step Domain Modeling Process

Move from domain information to a reviewed UML class diagram without mixing discovery and drawing too early.

LEARNING FOCUS

Apply the lecture's five-step workflow and distinguish domain discovery from UML visualization.

CORE LECTURE

Five steps from reality to a domain model

1 · Collect

Collect information about the application domain.

2 · Brainstorm

List candidate concepts and relevant domain knowledge.

3 · Classify

Organize and classify the brainstorming results.

4 · Visualize

Represent the domain model using a UML class diagram.

5 · Review

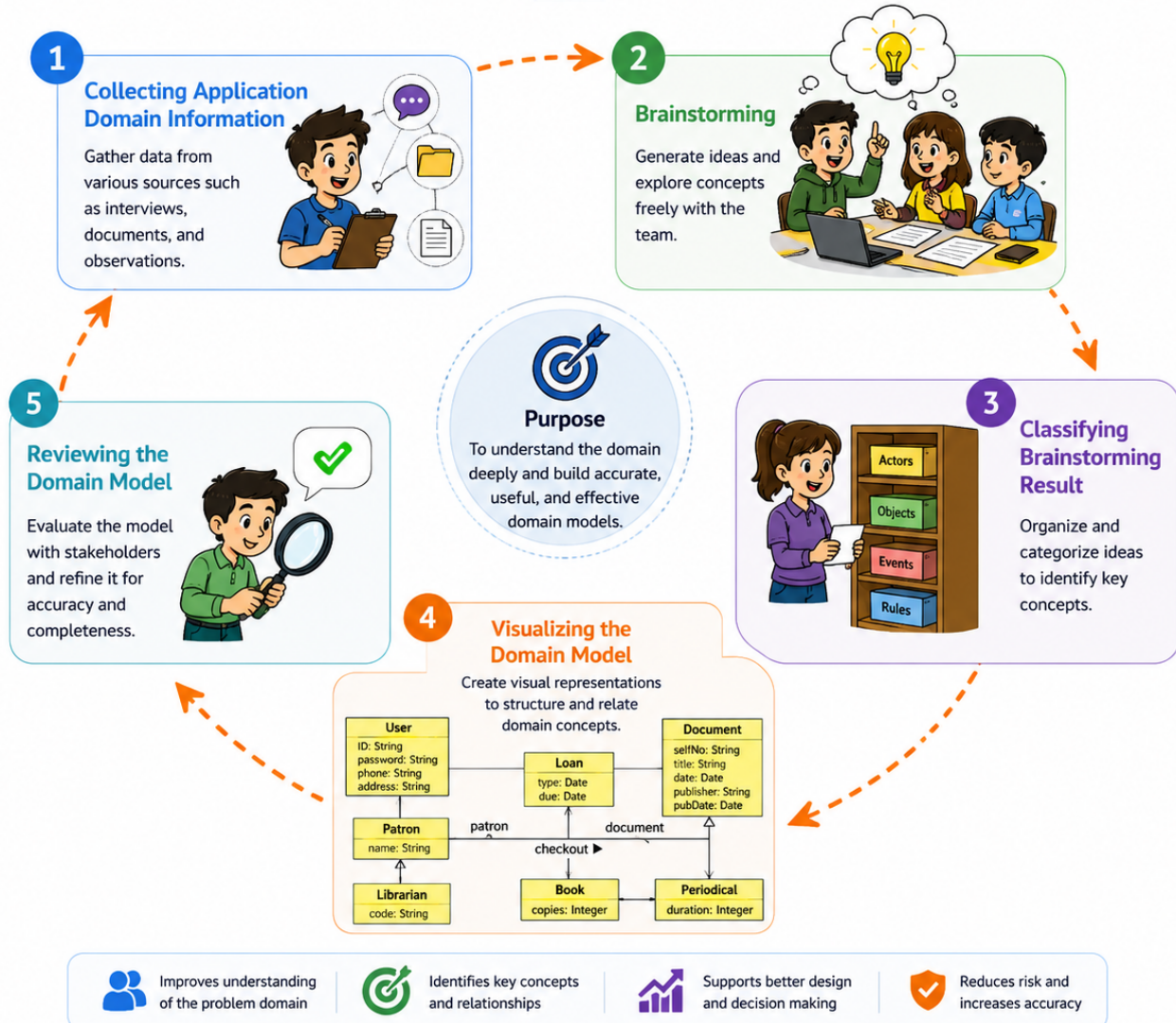
Inspect the model, obtain feedback, and correct it.

LECTURE VISUAL · PROCESS ANCHOR

See the complete workflow as one mental model

Domain Modeling Steps

A systematic approach to understand and model the application domain.



Copyright © The McGraw-Hill Companies, Inc.

The important boundary is between **discovering domain knowledge** and **drawing UML**. UML is the visualization step, not the brainstorming method.

LECTURE PRACTICE

Do not brainstorm and draw at the same time

The lecture explicitly warns that mixing brainstorming with diagram drawing can produce poor results. A practical team workflow is:

✓ List domain concepts and classify them on a whiteboard.

✓ Capture the brainstorming result so the team retains the raw domain knowledge.

- ✓ Have one or two members convert the classified result into a UML class diagram.
- ✓ Circulate the diagram to the team for review.
- ✓ Modify the model to reflect corrections and comments.

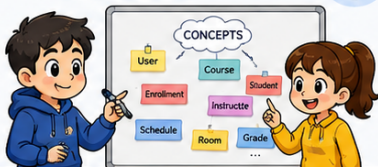
Tip for Domain Modeling



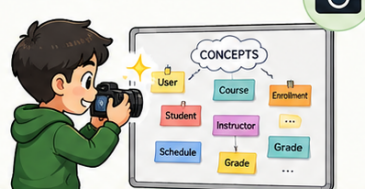
Do not do brainstorming and drawing at the same time.
The result could be very poor.



- 1 Team brainstorming:**
List the **concepts**, and then classify them on a whiteboard.



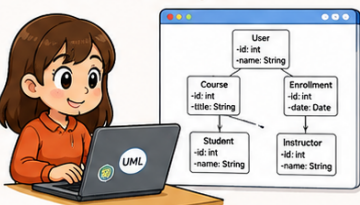
- 2** Take a picture(s) of the **whiteboard** using a digital camera.



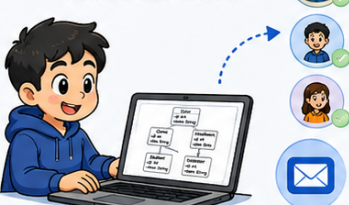
- 3** Email the digital images to team members.



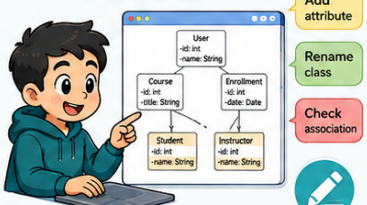
- 4** Have a member or two to convert the result to a **UML class diagram**.



- 5** Email the **UML** class diagram to all members to review.



- 6** Modify the diagram to reflect corrections and comments.



Follow this sequence for clear and effective domain modeling.



Better Team Alignment



Clear Communication



Higher Quality Design



Fewer Errors & Rework

Copyright © The McGraw-Hill Companies, Inc.

12

REMEMBER

DISCOVER → CLASSIFY → VISUALIZE → REVIEW. Do not let UML notation control the brainstorming session.

TOPIC 07 · LESSON 04 · 4 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 04

OO Foundations: Objects, Classes, Attributes & Operations

Domain modeling uses object-oriented concepts to describe the problem world before detailed software design begins.

LEARNING FOCUS

Distinguish object, class, attribute, operation, and encapsulation and explain why objects are the starting point in the OO paradigm.

OO PARADIGM

The world is viewed as interacting objects

**Object**

An instance of a class; a concrete entity in the modeled domain.

**Class**

A type: an abstraction and intentional definition of objects with similar properties and behavior.

**Attribute**

Defines a property of a class of objects.

**Operation**

Defines behavior of a class of objects.

Class $\neq$ Object

Class**Student**

A type describing what Student objects have in common.

Object**student #2339663**

One particular instance of the Student class.

Attributes

id, name, program

Operations

Behavior belongs to the class concept, but domain models normally omit operations.

ENCAPSULATION**Keep properties and behavior together**

The lecture defines **encapsulation** as defining/storing together the properties and behavior of a class or object. This helps us think about a class as one coherent concept rather than disconnected data and functions.

REMEMBER

OBJECTS EXIST; CLASSES DESCRIBE MEANINGFUL KINDS OF OBJECTS. In the domain model we focus on domain classes, their attributes, and their relationships.

TOPIC 07 · LESSON 05 · 5 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 05

UML Class Notation & Types

Learn how UML writes a class, its attributes and operations—and what must be omitted when the diagram is a domain model.

LEARNING FOCUS

Read compact and expanded class notation and interpret attribute/operation type syntax.

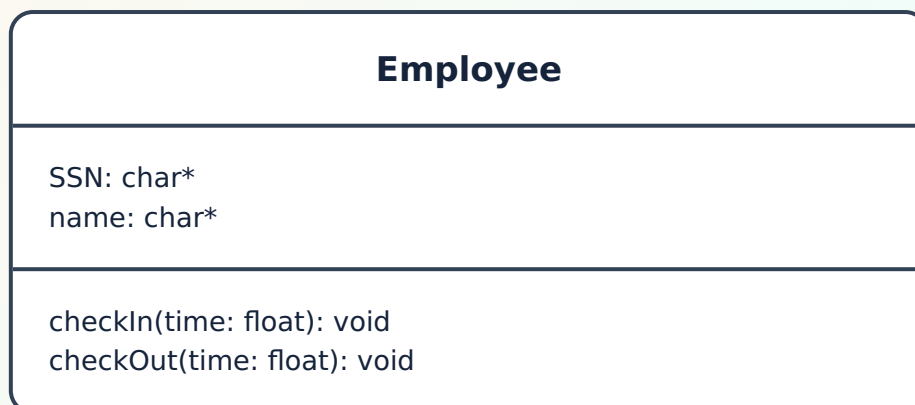
STRUCTURAL DIAGRAM

A UML class diagram shows structure

The lecture describes the UML class diagram as a **structural diagram** showing classes, their attributes and operations, and relationships between classes. For a **domain model**, show the classes, attributes and relationships—but **omit operations**.

VISUAL HOOK

Read the three compartments

**1****Class name**

The type being modeled.

2**Attributes**

Properties written using

`name : type .`

3

Operations

Behavior written as function name, parameters, parameter types, and return type.

TYPE SYNTAX**Decode UML type notation**

Notation	Meaning
<code>SSN: char*</code>	Attribute name + attribute type
<code>checkIn(time: float): void</code>	Function name + parameter name/type + return type

EXAM DISTINCTION

A full UML class diagram may contain operations. A DOMAIN MODEL CLASS DIAGRAM DOES NOT SHOW OPERATIONS because its purpose is conceptual understanding of the application domain.

TOPIC 07 · LESSON 06 · 6 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 06

Discovering Classes: Object or Attribute?

A noun phrase is not automatically a class. The real question is whether it has independent existence in the application domain.

LEARNING FOCUS

Apply the lecture rules for deciding whether a candidate noun/noun phrase should become a class or an attribute.

DECISION HOOK**Ask: can it exist independently?****Independent existence?**

If the concept can exist meaningfully on its own in the domain, it may be an object/class.

**Describes something?**

If it mainly describes another object or stores its state, it is probably an attribute.

**Can you enter its value?**

The lecture states that an attribute value can be entered directly, while an object itself is not entered as a value.

**Must it be created?**

Objects must be created by invoking a constructor, explicitly or implicitly. This is another practical clue that distinguishes an object from an attribute value.

LECTURE RULE**Attribute values describe; objects exist**

- ✓ **Attributes** describe objects or store state information about objects.

- ✓ You can enter an **attribute value** from the keyboard, but you do not “enter an object” as a simple value.

- ✓ **Objects** are created by invoking a constructor, explicitly or implicitly.

LECTURE EXAMPLE

“Number of seats” — class or attribute?

× Treat it as a class

“Number of seats” has no independent existence. It only makes sense as the number of seats of a car, airplane, or classroom.



✓ Treat it as an attribute

`numberOfSeats` belongs to a class such as Car, Airplane, or Classroom.

CANDIDATE TRIAGE

Classify noun phrases before drawing

Candidate	Likely result	Reason
Student	Class	Has independent existence in a university domain.
Course	Class	Represents a domain concept with its own identity and relationships.
Student name	Attribute	Describes a Student.
Number of seats	Attribute	Cannot stand meaningfully without another object.

REMEMBER

NOUNS GENERATE CANDIDATES, NOT ANSWERS. Use independent existence, descriptive role, and the lecture’s value-versus-object creation rule to decide whether a candidate belongs as a class or an attribute.

TOPIC 07 · LESSON 07 · 7 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 07

Inheritance, Generalization & Polymorphism

Use inheritance only when the domain truly contains an IS-A relationship—not merely because two concepts are related.

LEARNING FOCUS

Recognize generalization/specialization, apply the lecture's IS-A and conformance tests, and connect inheritance to information hiding and polymorphism.

INHERITANCE

Generalization ↔ specialization

▲ Vehicle

Superclass: more general.

Car

Subclass: more specific. A Car **is a** Vehicle.

Boat

Another specialized form of Vehicle.

Features defined for the superclass are automatically defined for subclasses. UML uses a hollow triangle pointing toward the superclass.

TWO LECTURE TESTS

Do not accept inheritance until it passes both tests



IS-A test

Every instance of the subclass must also be an instance of the superclass.



Conformance test

Relationships of the superclass must also make sense for subclasses.

COUNTEREXAMPLE

Why “Car is an Engine” fails

A Car contains or uses an Engine, but a Car is not an Engine. That is not inheritance. Likewise, the lecture’s Visiting Professor example shows that a proposed subclass can fail the conformance test when superclass relationships do not apply to it.

RELATED OO CONCEPTS

Inheritance, information hiding, polymorphism



Information hiding

Shield implementation detail to reduce change impact on other parts of a program.



Polymorphism

One thing can assume different forms; the lecture illustrates this with overloading.

REMEMBER

Inheritance means IS-A. If the relationship is “has-a,” “uses,” “works with,” or “contains,” another UML relationship is probably more appropriate.

TOPIC 07 · LESSON 08 · 8 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 08

Association, Roles & Direction

Association is the general UML relationship for domain concepts that participate in an application-specific relationship.

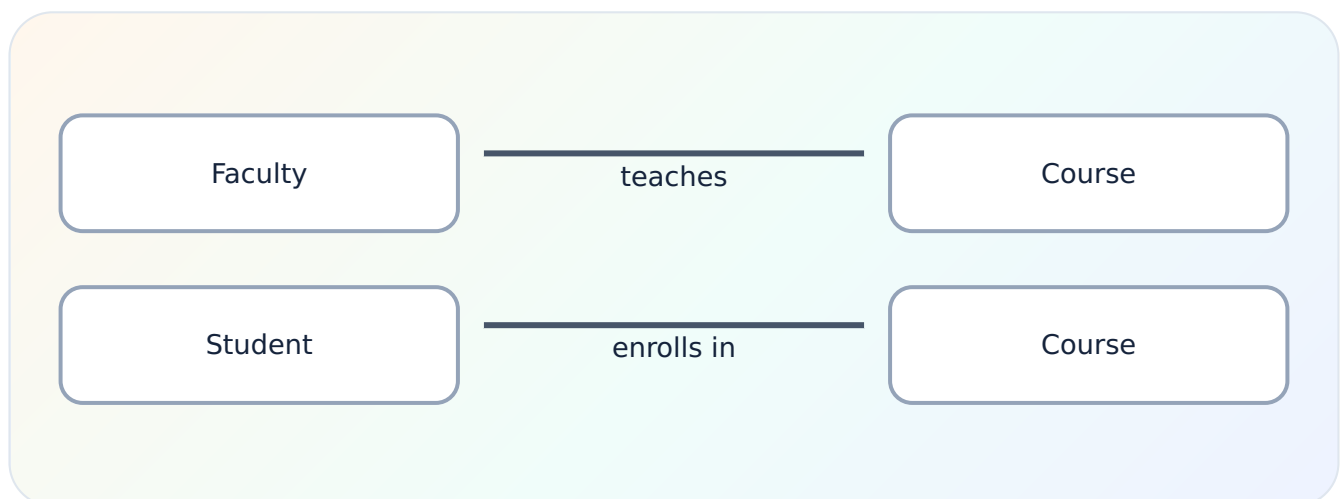
LEARNING FOCUS

Identify associations and read association names, role names, and direction without confusing association with inheritance or aggregation.

ASSOCIATION

A general relationship between concepts

The lecture defines association as a general, application-specific relationship. Examples include **Instructor teaches Course** and **User has Account**.

**READ THE LINE**

Name, direction, and roles answer different questions

Element	Question it answers	Example
Association name	What relationship exists?	teaches / enrolls
Direction	How should the relationship phrase be read?	Faculty → Course
Role name	What role does a participant play?	instructor / course

QUICK CLASSIFICATION

Association or something stronger?**Association**

“Faculty teaches Course” — a domain-specific relationship.

**Not inheritance**

A Faculty member is not a Course.

**Not aggregation**

Neither concept is modeled as a part of the other.

REMEMBER

Use association when two concepts are meaningfully related but the relationship is not an IS-A or whole-part relationship.

TOPIC 07 · LESSON 09 · 9 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 09

Multiplicity & Relationship Constraints

Multiplicity turns a relationship line into a precise statement about how many instances may participate.

LEARNING FOCUS

Read and write UML multiplicity constraints such as 1, 0..1, *, 1..*, m..n, and exact n.

LECTURE EXAMPLE

Base Station services Mobiles



Read it as: **one base station services zero or more mobiles, and every mobile is serviced by exactly one base station.**

MULTIPLICITY VOCABULARY

Decode the symbols

Notation	Meaning
1	Exactly one; also the default when omitted.
0..1	Zero or one.
* or 0..*	Zero or more.
1..*	One or more.
m..n	From m to n.
n	Exactly n.

HOW TO READ

Read multiplicity from the opposite end



Employee supervises Employees

A supervisor may supervise zero or more workers.



Always verbalize

Turn the diagram into a complete sentence; if the sentence sounds wrong, the multiplicity probably is.

REMEMBER

Multiplicity is a **CONSTRAINT on an association. It does not create a new relationship type; it makes an existing relationship more precise.**

Aggregation vs Composition

Both are whole-part relationships, but composition expresses stronger ownership and shared lifecycle.

LEARNING FOCUS

Distinguish aggregation from composition and place the diamond on the whole/aggregate side correctly.

AGGREGATION

Part-of / has-a

The lecture defines aggregation as a special form of association expressing that one object is part of another object. Example: an **Engine is part of a Car**. UML uses a **clear/white diamond** at the aggregate side.

◇ Car — Engine

Engine is part of Car.

COMPOSITION

A stronger whole-part relationship

◆ Book — Page

Composition is stronger than aggregation. The parts live and die with the whole.

Exclusive ownership

The lecture describes composition as an aggregated object exclusively owning the part.

COMPARE

White diamond vs black diamond

Question	Aggregation ◇	Composition ◆
Basic meaning	part-of / has-a	strong whole-part
Ownership	weaker	exclusive/stronger
Lifecycle	part may be independent	parts live and die with whole
Notation	white diamond	black diamond

Notation trap

The diamond goes on the **whole/aggregate side**, not on the part side.

REMEMBER

Do not use aggregation/composition merely because one class “uses” another. The relationship must actually mean whole-part.

TOPIC 07 · LESSON 11 · 11 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 11

Choosing the Right Class Relationship

The diagram becomes meaningful only when each line expresses the correct domain semantics.

LEARNING FOCUS

Choose among inheritance, association, aggregation, and composition using semantic tests rather than visual preference.

RELATIONSHIP MAP

Ask what the sentence means

Inheritance ▲

IS-A

Car is a Vehicle.

Association —

General relationship

Employee works on Project.

Aggregation ◇

PART-OF / HAS-A

Engine is part of Car.

Composition ◆

Strong whole-part

Part lifecycle depends on whole.

DECISION SEQUENCE

Use semantics before notation

Ask	If yes	Relationship
Is every instance of A also an instance of B?	A is a specialized B	Inheritance
Is A genuinely part of B?	Whole-part exists	Aggregation / Composition
Does the part live and die with the whole / is ownership exclusive?	Strong whole-part	Composition
Are the concepts simply related in the application?	Application-specific link	Association

ANNOTATED REFERENCE

Read a complete UML domain diagram

This annotated Library example brings many notations together in one view: abstract and stereotyped classes, attributes, enumeration, generalization, association, multiplicity, aggregation, composition, interface realization, usage dependency, and reading direction.

External source visual referenced by the course page

Reference: uml-diagrams.org — Domain diagram overview. Use it as a visual index: identify each notation first, then explain its semantic meaning.

Source: <https://images.weserv.nl/?url=www.uml-diagrams.org/class-diagrams/domain-diagram-overview.png>

Study strategy: do not memorize the red labels as isolated symbols. Cover the labels and try to identify each notation from the diagram itself, then state what it means in the Library domain.

[Open the original UML Class and Object Diagrams Overview ↗](#)

LECTURE SCOPE

Keep the domain model focused

The lecture notes that UML contains more relationship types, but for this domain-modeling material the focus is on **inheritance**, **association**, **aggregation**, and **composition**, with multiplicity and roles used to make associations precise.

MEMORY HOOK

**IS-A → INHERITANCE. PART-OF → AGGREGATION/COMPOSITION. OTHERWISE
MEANINGFUL DOMAIN LINK → ASSOCIATION.**

TOPIC 07 · LESSON 12 · 12 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 12

Worked Example: Online Program Application Domain

Read a domain story, extract the concepts, classify their properties, then connect them using the relationships taught in the lecture.

LEARNING FOCUS

Apply class discovery, attributes, associations, roles, and multiplicity to the lecture's sample program/application domain.

DOMAIN STORY

What exists in this world?

Users

Users search for programs. Staff users and administrators perform management tasks.

Programs

Programs have properties such as type, academic subject, term, language, country, deadlines, fees, and financial-aid information.

Applications

Students submit online applications for programs.

Feedback

Students can submit feedback associated with the domain.

STEP 1 · CANDIDATE CLASSES

Extract concepts, not every noun

**User**

Domain participant with identity and account-related properties.

**Program**

A study program with its own properties.

**Online Application**

An application submitted by a student.

**Feedback**

A domain object with description and relationships.

**Administrator**

A specialized/management role in the sample model.

STEP 2 · PUT PROPERTIES WHERE THEY BELONG

Examples of attributes

Class	Representative attributes from the lecture model
User	Name, Address, Phone No, e-mail ID, User Name, Password, Privilege

Class	Representative attributes from the lecture model
Program	Program Type, Academic Subject, Term of Study, Language of Instruction, Country, Region, Application Deadline, Fees, Scholarships & Financial aid
Online Application	Program Name, Student Name, Student ID, Address, Course
Feedback	Description

STEP 3 · CONNECT CONCEPTS

Relationships turn a list into a model

Student — submits — Application

Use association and multiplicity to express participation.

Application — for — Program

The application is linked to the program it targets.

Administrator — manages — Program

Management is modeled as a meaningful association.

Student — submits — Feedback

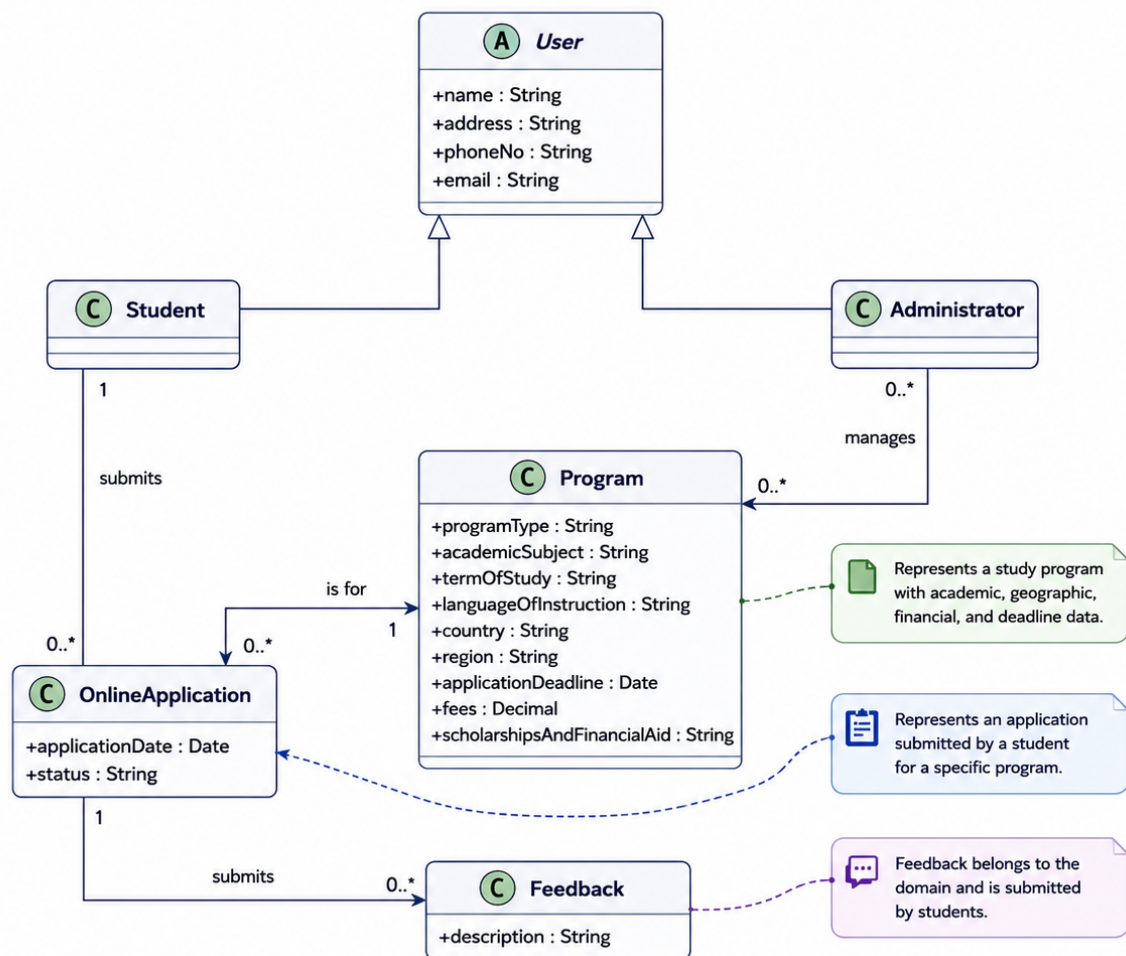
Another application-specific relationship.

STEP 4 · FINAL DOMAIN CLASS DIAGRAM

Compare your reasoning with the completed UML model

The image below is the completed solution supplied for this worked example. Read it after working through the earlier steps: identify the classes first, inspect their attributes, then trace each relationship and its multiplicity.

Domain Model — Program Application System



Modeling Principles Applied

✓ Model only what the evidence supports

✓ Avoid duplication (use relationships)

✓ Keep domain concepts at the right level

✓ Clear, meaningful relationships

Completed solution: **papclassuml.png**. Use it to verify the concepts, attributes, associations, and multiplicities derived in the worked example.

STUDY METHOD

Do not memorize the sample diagram as a picture. Reconstruct it from **CONCEPTS → **ATTRIBUTES** → **RELATIONSHIPS** → **MULTIPLICITIES**, then use the completed diagram above to check your reasoning.**

Extra Practice Examples

Practice the modeling decisions—not just the final notation—using small domains and four real UML reference examples.

LEARNING FOCUS

Transfer the lecture rules to unfamiliar domains by identifying classes, attributes, relationship semantics, and multiplicities.

EXTERNAL STUDY GALLERY · UML-DIAGRAMS.ORG

Four class diagrams to read, compare, and explain

The diagrams below are the original PNG examples from uml-diagrams.org, delivered through an image proxy to prevent the source site's anti-hotlink advertising response. The source links remain attached to every example.

External source visual: Library domain UML class diagram · <https://images.weserv.nl/?url=www.uml-diagrams.org/examples/class-example-library-domain.png>

Library Domain Model

01

Focus on Book/Book Item, Patron, Account, Library and Catalog. Study association, multiplicity, aggregation/composition, inheritance and enumeration.

[Open source](#) ↗

External source visual: Online shopping domain UML class diagram · <https://images.weserv.nl/?url=www.uml-diagrams.org/examples/class-example-online-shopping-domain.png>

Online Shopping Domain

02

Trace Customer, Account, Shopping Cart, Product, Order and Payment. Ask where ownership is modeled and why several relationships allow zero or many instances.

[Open source ↗](#)

External source visual: Bank account taxonomy UML class diagram · <https://images.weserv.nl/?url=www.uml-diagrams.org/examples/bank-account-domain-class-diagram.png>

External source visual: Hospital organization UML class diagram · <https://images.weserv.nl/?url=www.uml-diagrams.org/examples/class-example-hospital-organization.png>

Bank Account Taxonomy

03

Use this example to study generalization sets and specialization. Explain why different account classifications can coexist.

[Open source ↗](#)

Hospital Domain Model

04

Look for Person/Patient specialization, hospital staff structure and multiplicities. Translate each multiplicity back into a sentence about the real hospital.

[Open source ↗](#)

Reference gallery: [UML Class Diagrams Examples ↗](#)

HOW TO STUDY THE GALLERY

Use the same reading routine every time

- ✓ **Boundary:** state what domain the diagram is trying to understand.
- ✓ **Concepts:** identify the important classes and separate them from attributes.
- ✓ **Hierarchy:** find every IS-A relationship and test whether it is semantically valid.

- ✓ **Connections:** classify associations versus aggregation/composition.
- ✓ **Cardinality:** translate every multiplicity into a plain-English domain rule.
- ✓ **Explain:** summarize the model without pointing at notation—if you can explain the domain, you understand the diagram.

QUICK TRANSFER PRACTICE

Now model without a reference

University enrollment

Students enroll in sections taught by instructors; each section belongs to one course.

Vehicle domain

Test Vehicle/Car inheritance, Driver-Car association, and whether Engine is aggregation or composition.

Book publishing

Model Book, Author, Publisher and Page; justify each relationship rather than copying a familiar diagram.

Your challenge

Add multiplicities and explain every one as a sentence about the domain.

PRACTICE RULE

A good domain model is not the diagram with the most boxes. It is the smallest model that captures the important domain concepts, their properties, and their relationships clearly enough for shared understanding.

TOPIC 07 · LESSON 14 · 14 OF 14 LESSONS

TOPIC 07 · CHAPTER 5 · LESSON 14

Review, Agile Modeling & Exam Checklist

Finish the topic by connecting domain modeling to the wider methodology and checking whether the model is simple, correct, useful, and reviewable.

LEARNING FOCUS

Synthesize the topic, apply the lecture's agile guidance, and verify that you can read and construct a domain model from a short scenario.

APPLYING AGILE PRINCIPLES

Model only as much as the team needs

**Work closely with users**

Understand the application and application domain with customers and users.

**Keep it simple**

Perform domain modeling only if it is needed and expand the model incrementally.

**Model alongside other views**

Domain modeling may occur alongside actor-system interaction, object interaction, object state, and activity modeling.

TOPIC SYNTHESIS

Rebuild the story

1 · Understand

Why models and domain knowledge are needed.

2 • Discover

Collect → brainstorm → classify.

3 • Visualize

Classes + attributes + relationships.

4 • Refine

Inheritance, association, multiplicity, aggregation/composition.

5 • Review

Inspect, correct, simplify, iterate.

SOURCE COVERAGE CHECK**The complete Chapter 5 study boundary**

Lecture concept	Where you should now be confident
Why models are needed	Different perceptions, communication, and enhancement maintenance.
Domain modeling: What / Why / How	Common understanding, domain knowledge, classes/attributes/relationships, and UML visualization.
Five-step process	Collect → Brainstorm → Classify → Visualize → Review.
OO foundations	Class, object, attribute, operation, encapsulation.
Domain-model UML rule	Classes + attributes + relationships; operations are omitted.
Object vs attribute	Independent existence, state-description, value entry, and object creation via constructor.
Inheritance / generalization	IS-A, superclass/subclass, hollow triangle, conformance.
Association	Application-specific relationship, association name, direction, role.

Lecture concept	Where you should now be confident
Multiplicity	1, 0..1, *, 1..*, m..n, exact n and sentence-level interpretation.
Aggregation / composition	Part-of, white vs black diamond, exclusive ownership, lifecycle semantics.
Sample domain model	Reconstruct the Online Program Application model from concepts, attributes, relationships, and multiplicities.
Agile guidance	Work with users, keep modeling simple, expand incrementally, combine with other modeling views.

UML VIEWS IN CONTEXT

Know what each diagram answers

Diagram	Main lecture purpose
Class diagram	Information structure, relationships between data items, modular structure.
Use cases	User's view, functions, visual overview of main requirements.
Package diagram	Overall architecture and dependencies between components.
Sequence diagram	Individual scenario, interactions, sequence of messages.
Statechart	Responses to events and dynamic behavior.
Activity diagram	Business processes, concurrency/synchronization, task dependencies.

EXAM CHECKLIST

You are ready if you can do all of these

- ✓ Define a model and explain why software teams need models.
- ✓ State the five domain-modeling steps in order.
- ✓ Distinguish class, object, attribute, operation, and encapsulation.

- ✓ Explain why a domain model omits operations.
- ✓ Separate candidate classes from attributes using independent existence and the lecture's object-creation rule.
- ✓ Recognize inheritance and apply the IS-A/conformance tests.
- ✓ Identify association, role, direction, and multiplicity.
- ✓ Distinguish aggregation from composition and place the diamond correctly.
- ✓ Construct a small domain model from a scenario and justify each relationship.
- ✓ Read a complete diagram back into plain-English domain rules.

CONCEPT CHECK

Six quick questions

- ✓ Why should brainstorming be separated from UML drawing?
- ✓ Why is "number of seats" normally an attribute rather than a class?
- ✓ What extra clue does the lecture give for distinguishing an object from an attribute value?
- ✓ What two tests must inheritance pass?
- ✓ What is the meaning of `0..*` ?

- ✓ What extra semantic condition makes composition stronger than aggregation?

TOPIC 07 IN ONE SENTENCE

DOMAIN MODELING CONVERTS SHARED UNDERSTANDING OF THE PROBLEM WORLD INTO A REVIEWED UML CLASS DIAGRAM OF CONCEPTS, PROPERTIES, AND MEANINGFUL RELATIONSHIPS.